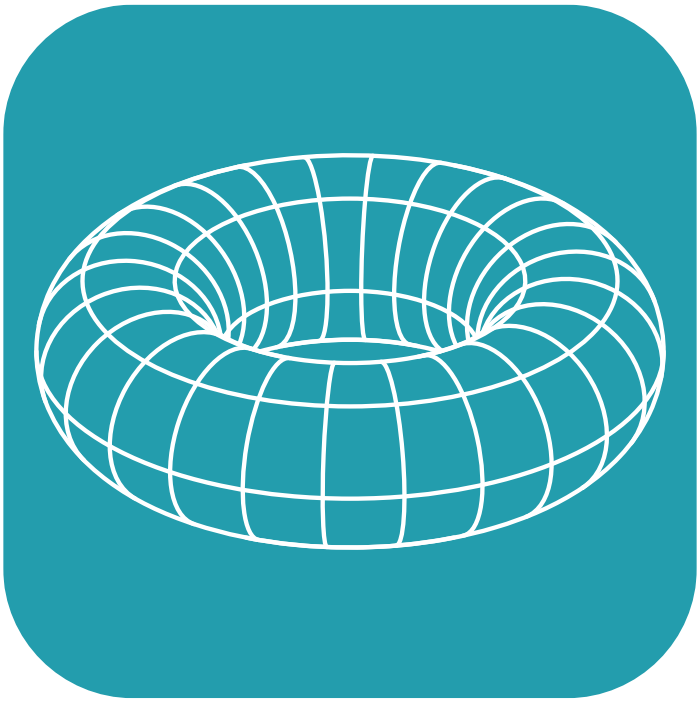


maquette

Render 3D models in Typst

Source	github.com/bernsteining/maquette
Typst Universe	typst.app/universe/package/maquette
Live demo	bernsteining.github.io/maquette

Version 0.2.1 · October 26, 2026

CONTENTS

Contents

Introduction . . . 3

Where to find sample models . . . 3

Quickstart . . . 3

- Inline Show Rule . . . 3

Config Reference . . . 4

Appearance . . . 5

- Custom Color . . . 5
- Background Color . . . 5

Camera Position . . . 6

- Cartesian coordinates . . . 6
- Spherical coordinates . . . 6
- Field of View . . . 7
- Framing: Zoom & Pan . . . 7

Projections . . . 8

- Perspective (default) . . . 8
- Orthographic . . . 8
- Isometric . . . 8
- Dimetric . . . 8
- Trimetric . . . 8
- Military . . . 8
- Cabinet . . . 9
- Cavalier . . . 9
- Fisheye . . . 9
- Stereographic . . . 9
- Curvilinear . . . 9
- Cylindrical . . . 9
- Pannini . . . 9
- Tiny Planet . . . 9

Shading & Lighting . . . 10

- Ambient & Light Direction . . . 10
- Hemisphere Ambient . . . 10
- Smooth Shading . . . 11
- Specular Highlights . . . 11
- Gamma Correction . . . 12
- Fresnel / Rim Lighting . . . 12
- Multi-Light . . . 13
- Area Lights . . . 13
- Tone Mapping . . . 14
- Shading Models . . . 14
  - Blinn-Phong (default) . . . 14
  - Unlit . . . 14
- Normal Mapping . . . 15
  - Flat . . . 15
  - Cel . . . 15
  - Gooch . . . 15
- Subsurface Scattering . . . 16
  - Without Subsurface Scattering . . . 16
  - Subsurface Scattering (back-lit bunny) . . . 16

Color Mapping . . . 17

- Curvature Map . . . 17
- Overhang Map . . . 17
- Scalar Function . . . 18

File Formats & Coloring . . . 19

- STL Per-face Color . . . 19
- PLY Format . . . 19
  - Meshes . . . 19
  - Point Clouds . . . 19
- OBJ Material Coloring . . . 20
- OBJ Textures . . . 20
- OBJ Groups . . . 21
  - Group Highlight . . . 21
    - Available Attributes . . . 21
  - Per-group appearance . . . 22
  - Annotations . . . 22

Render Modes . . . 23

- Solid (default) . . . 23
- X-Ray Mode . . . 23
- Wireframe . . . 24
- Solid + Wireframe . . . 24

Shadows . . . 25

- Ground Shadow . . . 25
- Cast Shadows . . . 25

Post-Processing . . . 26

- Antialiasing . . . 26
- Silhouette Outlines . . . 27
- Sharpening . . . 28
- Bloom & Glow . . . 28
- Ambient Occlusion . . . 29
- Depth Fog . . . 29

Effects . . . 30

- Clipping . . . 30
- Exploded View . . . 31
- Decimation . . . 32

Multi-View . . . 33

- Multi-View Grid . . . 33
- Turntable . . . 33

Debug . . . 34

Model Info & Measurements . . . 34

Models Credits . . . 35

Contributing . . . 35

Introduction

maquette is a Typst plugin for rendering 3D models directly inside your documents. It loads STL, OBJ, and PLY files and produces publication-ready images: no screenshots, no manual exporting. Everything runs with WASM.

Under the hood, maquette is a small rasterizer with a real lighting pipeline: multi-light Blinn-Phong shading, Fresnel reflections, subsurface scattering, ambient occlusion, outlines, depth fog, bloom, tone mapping, and more. Models can be rendered to PNG (rasterized, constant-size output) or SVG (scalable vector polygons). The full configuration: camera, lights, materials, post-processing lives in your `.typ` source, so every view is reproducible and version-controllable.

Where to find sample models

The examples below use small classics from a few free repositories. Any STL, OBJ or PLY file works.

- Stanford 3D Scanning Repository: the canonical bunny, dragon and happy Buddha, in PLY.
- Sketchfab (downloadable filter): the largest general library; most exports include OBJ.

Quickstart

Import a render function, read a model file, and call it. That's it.

```
#import "@preview/maquette:0.2.1": render-stl

#let cube = read("data/cube.stl", encoding: none)
#render-stl(cube)
```



For STL & PLY: Always read with `encoding: none`.

Here the cube is facing the camera, so it draws a square.

We'll see later how to tweak camera parameters to play with perspective.

The default output is PNG; use `format: "svg"` for vector output: `#render-stl(cube, format: "svg")`

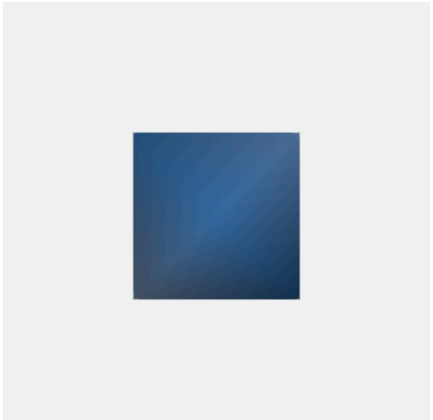
Inline Show Rule

With a show rule, you can write OBJ / STL / PLY geometry directly in fenced code blocks and have it rendered inline:

```
#show raw.where(lang: "obj"): it => {
  render-obj(bytes(it.text))
}
```

Here we're drawing a square pyramid inline, hence the different aspect because of how light behaves on the model's surface.

````
```obj
v  0.0  1.0  0.0
v -0.5  0.0 -0.5
v  0.5  0.0 -0.5
v  0.5  0.0  0.5
v -0.5  0.0  0.5
f 1 3 2
f 1 4 3
f 1 5 4
f 1 2 5
f 2 3 4 5
```
````



Config Reference

All parameters are optional, defaults are shown below. Names are written in kebab-case; snake_case also works.

```
{ // ── Camera & Viewport ──────────────────────────────────────
  "camera": null,                          // Camera position in world space (Cartesian); overrides azimuth/elevation
  "azimuth": 0,                            // Spherical camera: horizontal angle in degrees
  "elevation": 0,                          // Spherical camera: vertical angle in degrees
  "distance": null,                        // Spherical camera: distance from center (null = 3× bounding radius)
  "center": [0, 0, 0],                     // Look-at target (overridden by auto-center)
  "up": [0, 0, 1],                         // Up direction vector
  "fov": 45,                               // Vertical FOV in degrees (perspective only)
  "projection": "perspective",             // "perspective", "orthographic", "isometric" ...
  "auto-center": true,                     // Auto-center on model bounding box
  "auto-fit": true,                        // Scale model to fill viewport
  "zoom": 1.0,                             // Multiplier on auto-fit scale (>1 zooms in)
  "pan": [0, 0],                           // Screen-space recentring [right, up] as viewport fraction
  "width": 500,                            // Output width in pixels
  "height": 500,                           // Output height in pixels
  "background": "#f0f0f0",                 // Background color (hex); none, "none" or "" = transparent
  // ── Appearance ──────────────────────────────────────────────
  "color": "#4488cc",                      // Model fill color (hex)
  "stroke": {"color": "none", "width": 0}, // Triangle edge stroke (or just "#hex")
  "light-dir": [1, 2, 3],                  // Key light direction (default follows the camera)
  "ambient": 0.15,                         // Ambient light intensity (0-1)
  "mode": "solid",                         // "solid", "wireframe", "solid+wireframe", "x-ray"
  "xray-opacity": 0.1,                     // Front-face opacity for x-ray mode (0-1)
  "cull-backface": true,                   // Back-face culling and one-sided lighting
  "wireframe": {"color": "", "width": 1.0}, // Wireframe edges (or just "#hex")
  "smooth": true,                          // Gouraud smooth shading (best with PNG)
  "specular": 0.2,                         // Specular highlight intensity (0-1)
  "shininess": 32,                         // Specular exponent (higher = tighter)
  "gamma-correction": true,                // Compute lighting in linear sRGB space
  "fresnel": {"intensity": 0.3, "power": 5}, // Fresnel rim lighting (or just 0.3)
  "sss": false,                            // true or {intensity, power, distortion}
  "opacity": 1.0,                          // Global opacity (0-1)
  "lights": [],                            // [{type, vector, color, intensity, size, cast-shadow}]
  "tone-mapping": {"method": "", "exposure": 1.0}, // HDR tone mapping (or just "aces")
  "shading": "",                           // "blinn-phong" (default), "gooch", "cel", "flat", "normal", "unlit"
  "gooch-warm": "#ffcc44",                 // Gooch warm tone color
  "gooch-cool": "#4466cc",                 // Gooch cool tone color
  "cel-bands": 4,                          // Number of cel-shading bands
  "materials": {},                         // OBJ material map: { "name": "#hex" }
  "mtl": "",                               // OBJ material library text (render-obj finds it from mtllib)
  "highlight": {},                         // OBJ group highlight: "#hex" or {color, specular, ...}
  // ── Annotations ─────────────────────────────────────────────
  "annotations": false,                    // true or {groups, color, font-size, offset}
  // ── Color Mapping ───────────────────────────────────────────
  "color-map": "",                         // "overhang", "curvature", "scalar", or "" (off)
  "color-map-palette": [],                 // Custom hex color gradient (curvature/scalar)
  "scalar-function": "",                   // Math expression for scalar mode: "sqrt(x*x+y*y+z*z)"
  "color-map-property": "",                // PLY vertex property to use as the scalar in scalar mode
  "vertex-smoothing": 4,                   // Smooth color values across vertices (0-4)
  "overhang-angle": 45,                    // Overhang threshold in degrees
  // ── Outlines ────────────────────────────────────────────────
  "outline": false,                        // true or {color, width, threshold}
  // ── Effects ─────────────────────────────────────────────────
  "ground-shadow": false,                  // true or {opacity, color}
  "shadows": false,                        // Cast/self shadows: true or {per-pixel, softness, color, omni, ...}
  "clip": null,                            // Cut plane: (a,b,c,d) or {from|axis|normal, depth, keep, cap, hatch}
  "explode": 0,                            // Exploded view factor
  "decimate": 0,                           // Mesh simplification 0-1 (higher = fewer triangles)
  "point-size": 0,                         // Point cloud: kNN radius, or splat radius if point-splat (0 = auto)
  "point-neighbors": 12,                   // Point cloud: neighbors per point (higher = fewer holes)
  "point-boundary": 60,                    // Point cloud: cut connections across a normal jump > this angle°
  "point-denoise": false,                  // Point cloud: edge-preserving colour clean-up for crisper regions
  "point-splat": false,                    // Point cloud: draw points as round splats instead of reconstructing
  "antialias": 4,                          // 0: off, 1: FXAA, 2-3: SSAA x2/x4, 4-5: FXAA + SSAA x2/x4
  "ssao": false,                           // true or {samples, radius, bias, strength, space}
  "bloom": false,                          // true or {threshold, intensity, radius}
  "glow": false,                           // true or {color, intensity, radius}
  "sharpen": false,                        // true or {strength} (default 0.5)
  "fog": false,                            // Depth fog: true or {intensity (0-100, default 50), color}
  // ── Multi-View ──────────────────────────────────────────────
  "views": null,                           // Named views: ["front", "right", "top", ...]
  "turntable": {"iterations": 0, "elevation": 40}, // Turntable views (or just 6)
  "grid-labels": true,                     // Show labels on multi-view grids
  // ── Diagnostics ─────────────────────────────────────────────
  "debug": false,                          // Overlay model metadata
  "debug-color": "#cc2222"                 // Debug text color
}
```

Appearance

Custom Color

Change the global color of the model by changing the `color` field.

```
#render-stl(cube,
  color: "#c0ffee",
  width: 70%,
)
```



Background Color

Set `background` to a hex color to fill the image background.

```
#render-stl(cube,
  center: (0.5, 0.5, 0.5),
  background: "#1a1a2e",
  width: 70%,
)
```



Set `background` to `none` (the string `"none"` and an empty string `""` work too) for a transparent background. The PNG carries a real alpha channel, so the model blends into the page.

```
#render-stl(cube,
  background: none,
  width: 70%,
)
```



The cube looks like a flat square from this angle. Let’s learn how to change the point of view.

Camera Position

Cartesian coordinates

Change the camera position and where it points to using cartesian coordinates with `camera:(x,y,z)` and `center:(x,y,z)`. As you may have noticed with previous examples, maquette finds the model's bounding box automatically and points the camera at its centre (that is `auto-center: true`, the default), so most of the time no `center` is needed. Set `auto-center: false` to aim at the explicit `center` you provide instead.

```
#let teapot = read("teapot.obj")

#render-obj(teapot,
  camera: (0, 0, 10),
  up: (0, 1, 0),
)
```



The `up` parameter defines which direction points "up" in the scene. The default is `(0, 0, 1)` (Z-up), which matches the convention used by most CAD software and STL files. OBJ files exported from Blender, game engines, or other Y-up tools typically need `up: (0, 1, 0)` to display correctly.

Spherical coordinates

Instead of placing the camera with Cartesian `(x, y, z)` coordinates, you can use `azimuth` (horizontal angle) and `elevation` (vertical angle) in degrees. This makes it much easier to orbit around a model: just change the angles. When either is set, they override the `camera` field. The `distance` is auto-computed from the bounding box unless specified.

```
#render-obj(teapot,
  up: (0, 1, 0),
  azimuth: 30,
  elevation: -10,
  distance: 10,
)
```



🎥 **Placing the camera, fast.** The live browser demo (https://bernsteining.github.io/maquette/) runs the identical code faster: drag to orbit interactively, scroll to zoom, then paste the generated snippet straight into your document!

Field of View

Two settings shape the perspective camera: `distance`, how far the camera sits from the model's centre (default: 3× the radius of its bounding sphere), and `fov`, the vertical angle the image covers, in degrees (default `45`).

With `auto-fit: true` (the default), the whole model always stays in the image: the view is never narrower than what it takes to show the model's bounding sphere from top to bottom. A smaller `fov` therefore enlarges the model only until that sphere fills the frame height; to go further, use `zoom` (below). `distance` changes the perspective itself: up close, nearby parts look much larger than distant ones, while from far away the model looks flatter. Set `auto-fit: false` to use `fov` exactly as given; nothing then keeps the model in frame, so parts of it can be cropped.



`default`



`fov: 20`



`distance: 6`



`auto-fit: false, fov: 25`

Framing: Zoom & Pan

`auto-fit` frames the model's bounding sphere, which is larger than a flat or elongated model, so the model often sits in generous margins. `zoom` and `pan` reframe the picture without moving the camera, so the perspective stays the same:

- **`zoom`** enlarges the image around its centre (default `1`; `2` doubles the size). Whatever no longer fits is cropped.
- **`pan: (x, y)`** then moves the model across the image, in fractions of its width and height (default `(0, 0)`): `(0.1, 0)` moves it right by a tenth of the width, and a positive `y` moves it up.



`zoom: 1.0 (default)`



`zoom: 1.45`



`zoom: 1.45, pan: (-0.12, 0)`

At `zoom: 1.45` the teapot is larger, but its spout and handle touch both sides. `pan: (-0.12, 0)` moves it left by 12% of the width, so the handle clears the right edge.

Projections

maquette supports 14 projection types. Set `projection: "name"` to switch.

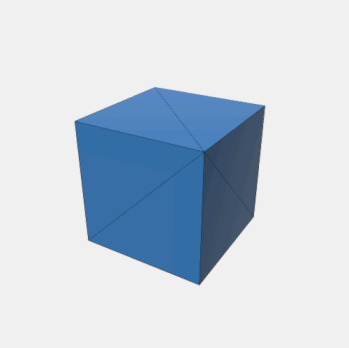
The parallel projections below draw triangle edges with `stroke: (color, width)` to show how each one treats the cube's lines. The wide-angle lenses stand in the middle of `city.obj`, a grid of towers, with a shared camera:

```
#let street = (camera: (0, 6, 0), center: (17, 2, 11), up: (0, 1, 0),
  auto-center: false, auto-fit: false, cull-backface: false)
```

Perspective (default)

Objects farther from the camera appear smaller, giving a natural sense of depth.

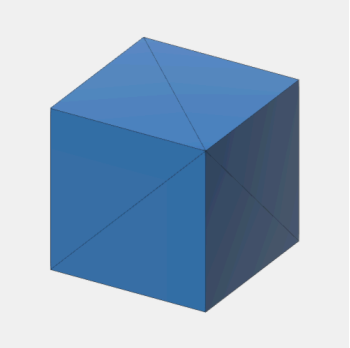
```
#render-stl(cube,
  camera: (3, 2, 2),
  stroke:
    (color: "#111111",
     width: 1.0),
)
```



Orthographic

No perspective foreshortening: parallel lines stay parallel regardless of distance.

```
#render-stl(cube,
  camera: (3, 2, 2),
  stroke:
    (color: "#111111",
     width: 1.0),
  projection: "orthographic",
)
```



Isometric

Three principal axes appear equally foreshortened. Common in technical illustrations and game art.

```
#render-stl(cube,
  camera: (3, 2, 2),
  stroke:
    (color: "#111111",
     width: 1.0),
  projection: "isometric",
)
```



Dimetric

Two axes equally foreshortened, the third differs. Elevation 20.7°, azimuth 45°.

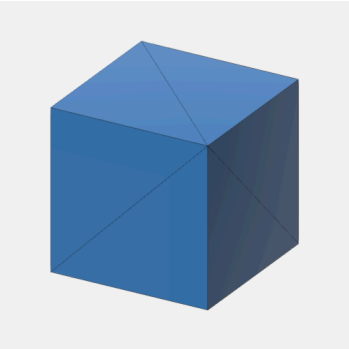
```
#render-stl(cube,
  camera: (3, 2, 2),
  stroke:
    (color: "#111111",
     width: 1.0),
  projection: "dimetric",
)
```



Trimetric

All three axes have different foreshortening. Elevation 25°, azimuth 30°.

```
#render-stl(cube,
  camera: (3, 2, 2),
  stroke:
    (color: "#111111",
     width: 1.0),
  projection: "trimetric",
)
```



Military

Oblique projection: the plan keeps its true shape, rotated 45°, and heights are drawn straight up at full scale.

```
#render-stl(cube,
  camera: (3, 2, 2),
  stroke:
    (color: "#111111",
     width: 1.0),
  projection: "military",
)
```



Cabinet

Oblique projection: front face at true shape, depth axis at half scale, 45° angle.

```
#render-stl(cube,
  camera: (3, 2, 2),
  stroke:
    (color: "#111111",
     width: 1.0),
  projection: "cabinet",
)
```



Cavalier

Like cabinet, but the depth axis is drawn at full scale at a 45° angle. All dimensions are preserved equally.

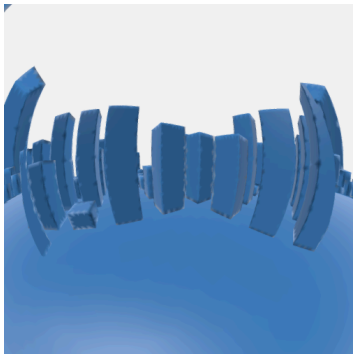
```
#render-stl(cube,
  camera: (3, 2, 2),
  stroke:
    (color: "#111111",
     width: 1.0),
  projection: "cavalier",
)
```



Fisheye

Equidistant: angular distance maps linearly to radius, so distortion is uniform across the field, which can exceed 180°.

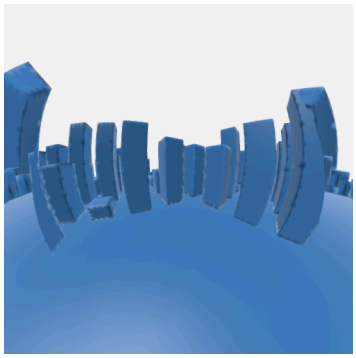
```
#render-obj(city, ..street,
  projection: "fisheye",
  fov: 200,
)
```



Stereographic

Conformal: preserves local shapes but enlarges the periphery. Compare the outer towers with fisheye.

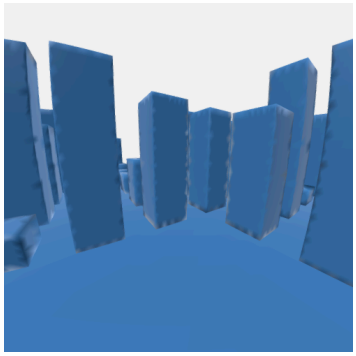
```
#render-obj(city, ..street,
  projection:
"stereographic",
  fov: 200,
)
```



Curvilinear

Perspective with barrel distortion: straight lines curve outward near the edges, simulating a wide-angle lens.

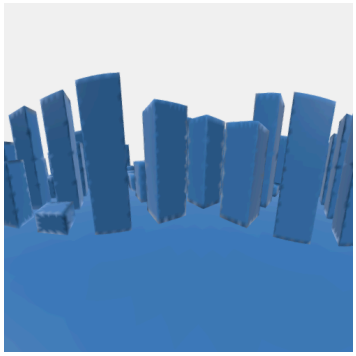
```
#render-obj(city, ..street,
  projection: "curvilinear",
  fov: 110,
)
```



Cylindrical

Horizontal angles map linearly, like a panorama, while vertical lines stay straight.

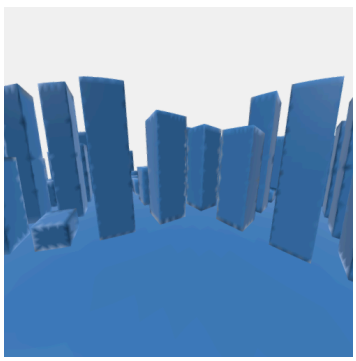
```
#render-obj(city, ..street,
  projection: "cylindrical",
  fov: 160,
)
```



Pannini

Architectural photography projection: verticals stay straight and horizontals curve gently. A hybrid between cylindrical and stereographic.

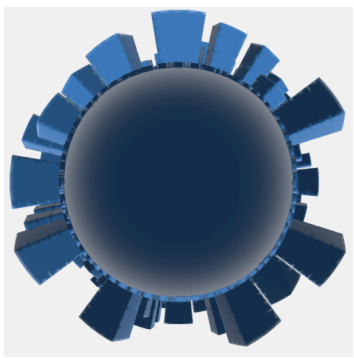
```
#render-obj(city, ..street,
  projection: "pannini",
  fov: 150,
)
```



Tiny Planet

Full 360° inverse projection: the camera looks straight up, so the ground wraps around the centre and the sky fills the outer ring. Backface culling is disabled automatically.

```
#render-obj(city,
  camera: (0, 4, 0),
  center: (0, 100, 0),
  up: (1, 0, 0),
  auto-center: false,
  auto-fit: false,
  fov: 210,
  zoom: 1.6,
  projection: "tiny-planet",
)
```



Shading & Lighting

Ambient & Light Direction

The `ambient` parameter (0–1) controls how much light reaches surfaces regardless of their orientation. Low values create dramatic contrast; high values flatten the shading. The `light-dir` vector points toward the light. When unset, it follows the camera: the light comes from above, slightly to one side of the viewer, whatever the model's orientation.

`ambient: 0.05`



`ambient: 0.3`



`ambient: 0.6`



Hemisphere Ambient

The `ambient` parameter accepts either a number or an object with `intensity`, `sky`, and `ground` fields. Hemisphere ambient lights surfaces facing up with the sky color and surfaces facing down with the ground color, blending the two on slopes. It imitates light bounced from the surroundings at no extra cost.

The default is `ambient: (intensity: 0.15, sky: "#ccd4e0", ground: "#d4ccc4")`.

Hemisphere ambient

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  specular: 0.3,
  ambient: (intensity: 0.4, sky: "#8899cc", ground: "#443322"),
  width: 100%,
)
```



Flat ambient

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  specular: 0.3,
  ambient: (intensity: 0.4, sky: "#ffffff", ground: "#ffffff"),
  width: 100%,
)
```



Smooth Shading

Smooth shading is enabled by default. Vertex normals are averaged across adjacent faces and lighting is interpolated per-pixel (Gouraud shading), smoothing out the faceted appearance. Set `smooth: false` to revert to flat shading, where each triangle gets a single color based on its face normal.

Smooth (default)



Flat (`smooth: false`)



Specular Highlights

Add Blinn-Phong specular highlights with the `specular` parameter (0–1). The `shininess` exponent controls how tight the highlight is: low values produce a broad, diffuse sheen; high values create a sharp, glossy spot. Specular works with both flat and smooth shading, and is most effective with PNG output.

Diffuse only



Specular (`specular: 0.6, shininess: 32`)



Gamma Correction

By default (`gamma-correction: true`), colors are converted to linear space before shading and back to sRGB afterward. This produces physically accurate lighting: midtones brighten, dark areas gain detail, and specular highlights blend smoothly. Disabling it (`gamma-correction: false`) computes lighting directly in sRGB, which is faster but produces harsher contrast and less natural results.

Without (`gamma-correction: false`)



With (default)



Fresnel / Rim Lighting

Fresnel rim lighting brightens edges where the surface curves away from the camera, making objects stand out from the background. Control with `fresnel: (intensity, power)` or just `fresnel: 0.6` for intensity only. Higher power gives a thinner rim. Works with both flat and smooth shading.

Without fresnel



With (`fresnel: 0.6`)



Multi-Light

By default, a single white directional light shines from `light-dir`. The `lights` array replaces it with any number of lights. Each light has a `type` (default `directional`), a `vector`, a `color`, an `intensity` and a `cast-shadow` flag. The type decides what `vector` means:

- **`directional`**: parallel rays, like sunlight. `vector` points toward the light.
- **`positional`**: a point light at position `vector`. Its brightness falls off with the square of the distance; `intensity` is the brightness at the model's center (or at the edge of its bounding sphere, for a light inside it), so the same value works in any units.
- **`area`**: a disk light at position `vector` with radius `size`, which softens its highlight and shadow, see Area Lights. It falls off with distance like a positional light, but shines only from the face turned toward the model's center: it weakens at grazing angles and lights nothing behind it.

Surfaces facing away from a light receive none of its direct light, only ambient. With `cull-backface: false`, which renders the inside of open meshes, surfaces are lit from both sides instead.

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.19,
  ambient: 0.05,
  specular: 0.5,
  color: "#cccccc",
  lights: (
    (type: "positional",
     vector: (3, 3, 0),
     color: "#ff4444",
     intensity: 1.2),
    (type: "positional",
     vector: (-3, 2, 2),
     color: "#44ff44",
     intensity: 1.0),
    (type: "directional",
     vector: (0, 1, 0),
     color: "#4444ff",
     intensity: 0.5),
  )
)
```



Area Lights

Set a light's `type` to `area` and give it a `size` to make it a **disk area light** rather than an infinitesimal point.



size: 0 (hard point)



size: 1.5



size: 4

As `size` grows, the sharp specular glint spreads into a soft sheen and shadow edges blur into penumbra. `size` is a per-light property, so a scene can mix a small key light with a large, soft fill:

```
lights: ((type: "area", vector: (4, 7, 5), size: 1.5,
          color: "#fff", intensity: 2.4),),
shadows: (per-pixel: true),
```

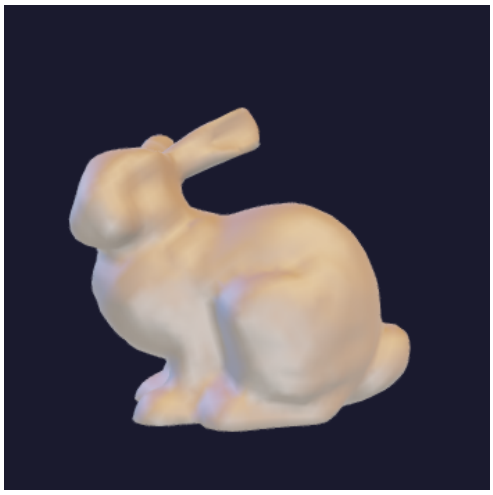
Tone Mapping

When multiple bright lights, strong specular, or fresnel push color values above 1.0, the default behavior hard-clips them to white, creating flat, washed-out highlights. Tone mapping compresses these HDR values gracefully, preserving detail and color in bright areas. Two operators are available: `"reinhard"` (simple, neutral) and `"aces"` (filmic, higher contrast). Use `tone-mapping: (method: "aces", exposure: 1.5)` for full control, or just `tone-mapping: "aces"` for the method alone.

No tone mapping



Reinhard



ACES



Shading Models

The `shading` parameter selects the lighting model, to configure the lights behaviour in the scene.

Blinn-Phong (default)

Photorealistic diffuse + specular.



Unlit

Base color only, no lighting.



Normal Mapping

Maps surface normals to RGB.



Flat

Face-normal shading. Disables `smooth` unless explicitly set.



Cel

Toon shading with discrete color bands.

Control steps with `cel-bands`.



Gooch

Warm-to-cool non-photorealistic shading.

Customize with `gooch-warm` and `gooch-cool`.



Subsurface Scattering

maquette approximates subsurface scattering with a cheap, view-dependent hack rather than true volumetric light transport: the *Approximating Translucency* technique (Barré-Brisebois & Bouchard, GDC 2011), a single dot product between the view direction and the back-facing light. It gives the warm glow of light passing through thin geometry (wax, skin, marble, leaves); back-lit areas glow with a color derived from the light and the model's base color. There's no real thickness sampling, so the glow is uniform rather than thickness-driven. Works with any shading model.

Without Subsurface Scattering

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  lights: (
    (type: "positional",
     vector: (-0.1, 0.14, -0.04),
     color: "#ff0000",
     intensity: 3.0),
  ),
)
```



Subsurface Scattering (back-lit bunny)

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  lights: (
    (type: "positional",
     vector: (-0.1, 0.14, -0.04),
     color: "#ff0000",
     intensity: 3.0),
  ),
  sss: (intensity: 4,
    power: 3.5,
    distortion: 0.2),
)
```



The `sss` dictionary has three knobs: **`intensity`** scales the overall glow; **`power`** sharpens its falloff, so higher values let light show through only the thinnest parts (the ears here); and **`distortion`** wraps the transmitted light around the surface normal for a softer, broader spread.

Color Mapping

Color mapping replaces the uniform model color with a gradient derived from geometric properties.

Curvature Map

Colors vertices based on local surface curvature, the rate at which the surface bends. Low curvature (flat areas) maps to blue/dark colors, while high curvature (sharp edges, creases) maps to red/bright colors. Useful for quality inspection, identifying sharp features, or visualizing mesh topology.

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  ambient: 0.3,
  specular: 0.5,
  vertex-smoothing: 4,
  color-map: "curvature",
  width: 70%,
)
```



You might have noticed the appearance of a `vertex-smoothing` setting in the previous render. This parameter allows to smoothen the color distribution across vertices, otherwise the color looks flaky, as you can see with `vertex-smoothing: 0`:

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  ambient: 0.3,
  specular: 0.5,
  color-map: "curvature",
  vertex-smoothing: 0,
  width: 70%,
)
```



Overhang Map

Faces steeper than `overhang-angle` (relative to vertical) are highlighted in red, while supported faces remain green. This directly maps to where a 3D printer would need support structures.

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  ambient: 0.3,
  specular: 0.5,
  color-map: "overhang",
  overhang-angle: 45,
  width: 70%,
)
```



Scalar Function

Color vertices based on a user-defined mathematical function `f(x,y,z)`. The `scalar-function` expression is evaluated at each vertex position, producing scalar values that are automatically normalized to `[0, 1]` (min value → 0, max value → 1), then linearly interpolated through the `color-map-palette` color stops. If no palette is specified, the default blue → cyan → green → yellow → red gradient is used. Per-vertex colors are interpolated across triangle faces for smooth results.

The expression can use:

- **Variables:** `x`, `y`, `z` (vertex coordinates)
- **Constants:** `pi`, `e`, `tau`
- **Arithmetic:** `+`, `-`, `*`, `/`, `^` (power)
- **Comparison:** `<`, `>`, `<=`, `>=`, `==`, `!=` (return 0.0 or 1.0)
- **Logical:** `&&` (and), `||` (or), `!` (not)
- **Functions:**
  - Basic: `abs`, `sqrt`, `min`, `max`, `clamp`, `sign`, `pow`
  - Trigonometric: `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`
  - Hyperbolic: `sinh`, `cosh`, `tanh`
  - Exponential/Logarithmic: `exp`, `ln`, `log10`, `log2`
  - Rounding: `floor`, `ceil`, `round`, `fract`
  - Graphics: `step`, `smoothstep`, `mix`, `lerp`, `length`
  - Other: `mod`

Some examples:

```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  ambient: 0.3,
  specular: 0.5,
  color-map: "scalar",
  scalar-function: "smoothstep(-0.1, 0.1, x)",
)
```



```
#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  ambient: 0.3,
  specular: 0.5,
  color-map: "scalar",
  scalar-function: "sin(x*60)*cos(y*60) + sin(y*60)*cos(z*60) +
sin(z*60)*cos(x*60)",
  color-map-palette: (
    "#1a0533",
    "#6b2fa0",
    "#e85d75",
    "#ffcc33",
  ),
)
```



File Formats & Coloring

STL Per-face Color

Some binary STL files encode per-face colors in the attribute bytes using the RGB565 format. maquette detects and renders these automatically. When present, the `color` parameter is ignored in favor of the embedded colors.

```
#let colored = read("/examples/data/colored_cube.stl", encoding:
none)

#render-stl(colored, projection: "isometric", width: 32%)
```



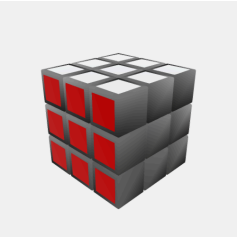
PLY Format

Meshes

maquette handles PLY files in ASCII and binary (little/big-endian) formats. PLY can store colors in its format, allowing us to color the model directly. Enjoy this beautiful PLY-colored Rubik's cube.

```
#let rubi = read("/examples/data/rubi_blender.ply", encoding:
none)

#render-ply(rubi,
  azimuth: 45,
  elevation: 25,
  distance: 8.7,
  width: 32%,
)
```



Point Clouds

PLY files can also contain clouds of points. 3D scanning apps usually allow to export in such a format.

By default it reconstructs a surface with k-NN. Set `point-splat: true` to instead draw each point **as a round splat**.

```
#let rubi_scan = read("rubi_scan.ply", encoding: none)

#render-ply(rubi_scan,
  point-neighbors: 10,
  point-boundary: 70,
  azimuth: -197,
  elevation: 20.5,
  up: (0, 1, 0),
  zoom: 2.5,
  width: 76%,
)
```



```
#let rubi_scan = read("rubi_scan.ply", encoding: none)

#render-ply(rubi_scan,
  point-splat: true,
  point-size: 0.01,
  azimuth: -197,
  elevation: 20.5,
  up: (0, 1, 0),
  zoom: 2.5,
  width: 76%,
)
```



OBJ Material Coloring

OBJ files can reference materials via `usemtl` directives. Provide a `materials` map to assign hex colors to each material name.

We list `cube.obj`'s materials as follows:

```
$ grep usemtl cube.obj
usemtl face1
usemtl face2
usemtl face3
usemtl face4
usemtl face5
usemtl face6
```

Then colorize material-wise:

```
#let obj-cube = read("/examples/data/
cube.obj")

#render-obj(obj-cube,
  camera: (3,3,3),
  materials: (
    face4: "#2ecc71",
    face5: "#222222",
    face6: "#ff2222",
  ),
)
```



OBJ Textures

An OBJ references image textures through a material library: the `.obj` names a `.mtl` with `mtllib`, and the `.mtl` points at a diffuse map with `map_Kd`. You never list these files yourself: give `render-obj` the **path** to the `.obj` plus a `read:` lambda and it **auto-discovers** the chain: it follows `mtllib` to the `.mtl`, then each `map_Kd` to its image, and reads every file for you.

Supported texture formats are **PNG, JPEG, TGA and BMP**.

```
#render-obj(
  "/examples/data/treasure-chest/chest.obj",
  read: p => read(p, encoding: none),
  up: (0, 0, 1),
  azimuth: 35,
  elevation: 20,
  width: 70%,
)
```



Every referenced path is resolved **relative to the `.obj`**, so the material and its textures live next to the model (or in whatever subfolder the paths spell out), here everything sits in one directory:

```
treasure-chest/
├── chest.obj          (mtllib: chest.mtl)
├── chest.mtl          (map_Kd: chest_diffuse.jpg)
└── chest_diffuse.jpg
```

The `read:` lambda is required because a Typst package cannot reach your project's files on its own. You provide the reader once, and maquette walks the whole chain for you. Textures modulate the lit surface, so lighting, ambient and shadows all still apply. They are sampled for raster (PNG) output, while SVG output falls back to the material's flat `Kd` colour.

OBJ Groups

Group Highlight

OBJ files with `g` or `o` directives define named groups. The `highlight` map assigns custom styling to specific groups.

Groups not listed keep their default appearance.

Available Attributes

When using the full object syntax, all attributes are optional:

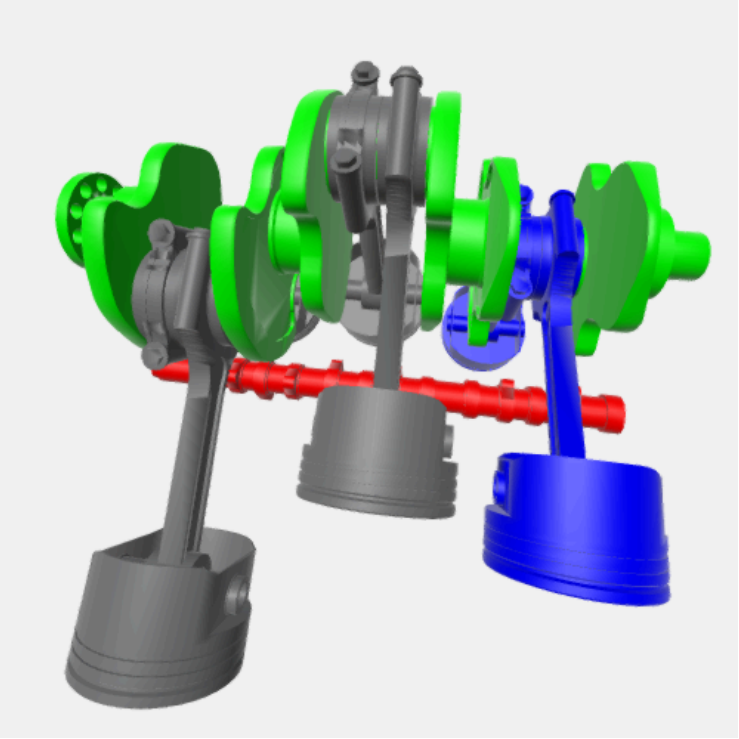
```
"highlight": (
  "GroupName": "#c0ffee",    // Simple: just a color string
  "AnotherGroup": (          // Advanced: full appearance object
    "color": "#ff0000",      // Hex color (overrides global color)
    "specular": 0.8,         // Specular intensity 0-1 (overrides global)
    "shininess": 64,         // Specular exponent (overrides global)
    "ambient": 0.3,          // Ambient light 0-1 (overrides global)
    "stroke": "#000000",     // Triangle edge stroke color
    "stroke-width": 1.0,     // Triangle edge stroke width
    "opacity": 0.5,          // Transparency 0-1 (0=invisible, 1=opaque)
    "label": "Piston A",     // Annotation text (default: the group name)
    )
)
```

We can list the groups as follows:

```
$ grep "g " crankshaft.obj
g Model__Piston_F
g Model__Head
g Model__Piston_E
g Model__Head_2
g Model__Piston_D
g Model__Head_3
g Model__Piston_C
g Model__Head_4
g Model__Piston_B
g Model__Head_5
g Model__Piston_A
g Model__Head_6
g Model__Crankshaft
g Model__Camshaft
```

Here's an example of a crankshaft with several parts defined by groups in the `.obj` file format:

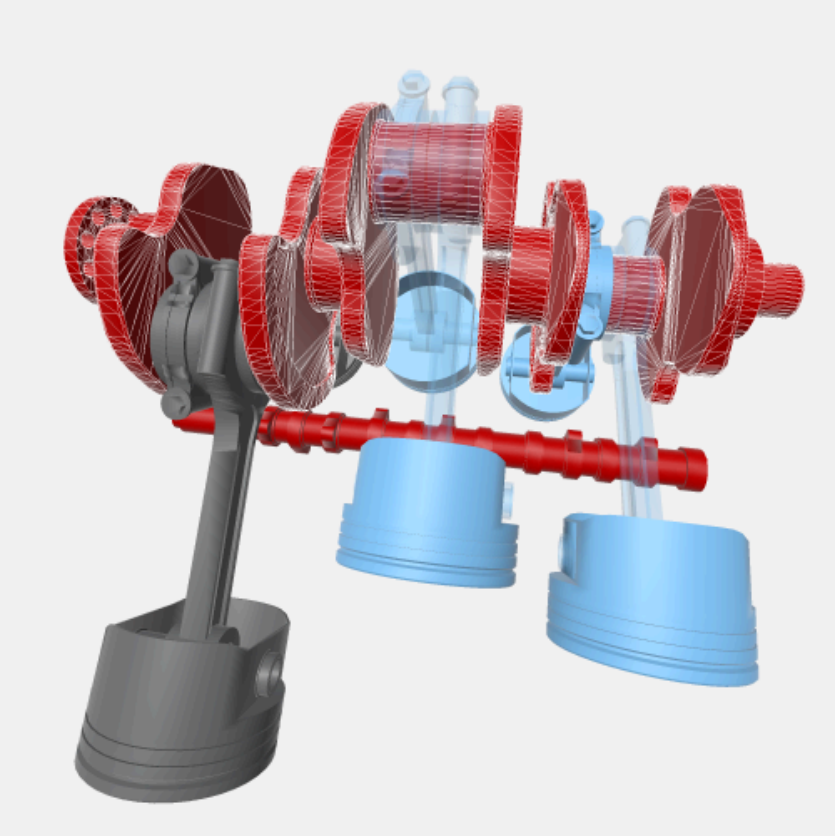
```
#align(center,
render-obj(crankshaft,
  camera: (-100, -100, 500),
  up: (0, -1, 0),
  zoom: 1.25, pan: (0, 0.08),
  color: "#777777",
  highlight: (
    Model__Camshaft: "#ff0000",
    Model__Crankshaft: "#00ff00",
    Model__Piston_B: (color: "#0000ff"),
  ),
))
```



Per-group appearance

Instead of a plain color, pass a dictionary with specific appearance overrides to selectively change parts appearance.

```
#render-obj(crankshaft,
  camera: (-100, -100, 500),
  up: (0, -1, 0),
  zoom: 1.25, pan: (0, 0.08),
  color: "#777777",
  antialias: 3,
  highlight: (
    Model__Crankshaft:
      (color: "#cc0000",
       stroke: "#ffffff",
       stroke-width: 0.5),
    Model__Piston_A: (color: "#88ccff", opacity: 0.3),
    Model__Piston_C: (color: "#88ccff", opacity: 0.3),
    Model__Piston_D: (color: "#88ccff", opacity: 0.3),
  ),
)
```

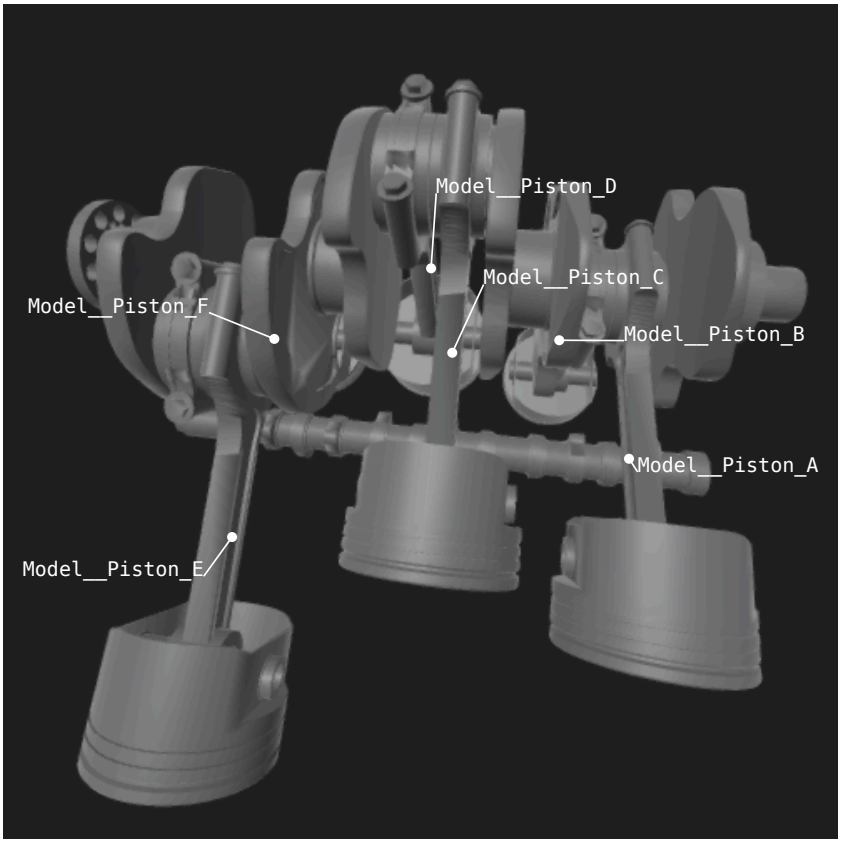


Annotations

Annotate OBJ groups by drawing a leader line from each group's centroid to a text label.

Pass `annotations: true` to label all groups with default styling, or pass an object to customize. The `groups` field filters to specific groups; `color`, `font-size`, and `offset` control appearance. A group's `label` in `highlight` replaces its name in the annotation text, while `groups` still matches the group name.

```
#render-obj(crankshaft,
  camera: (-100, -100, 500),
  up: (0, -1, 0),
  zoom: 1.25, pan: (0, 0.08),
  color: "#777777",
  annotations: (
    groups:
      ("Model__Piston_A",
       "Model__Piston_B",
       "Model__Piston_C",
       "Model__Piston_D",
       "Model__Piston_E",
       "Model__Piston_F"),
    color: "#ffffff",
    font-size: 12,
    offset: 45,
  ),
  background: "#222222"
)
```



Render Modes

Solid (default)

Nothing new, since it's the default mode we saw earlier, but it allows me to introduce this beautiful low poly `brain_skull.obj`.

```
#let skull-brain = read("/examples/data/brain_skull.obj")

#render-obj(skull-brain,
  azimuth: 270,
  up: (0,1,0),
  distance: 200,
  color: "#e8e8e8",
)
```



X-Ray Mode

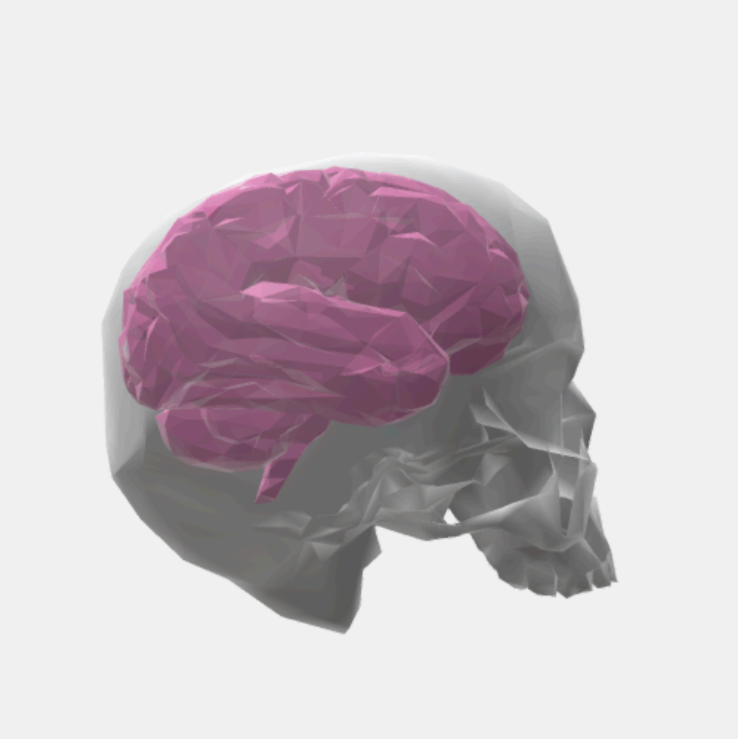
(`mode: "x-ray"`) creates transparent front-facing surfaces. Back-facing surfaces remain fully opaque.

Adjust transparency with `xray-opacity` (default 0.1 opacity).

Ideal for examining joints, internal components, nested geometry, and embedded models.

Our previous skull model now unveils its inner brain!

```
#render-obj(skull-brain,
  azimuth: 270,
  up: (0,1,0),
  distance: 200,
  highlight: (
    "Skull": (color: "#e8e8e8"),
    "Brain": (color: "#ee69b4"),
  ),
  xray-opacity: 0.3,
  mode: "x-ray",
)
```



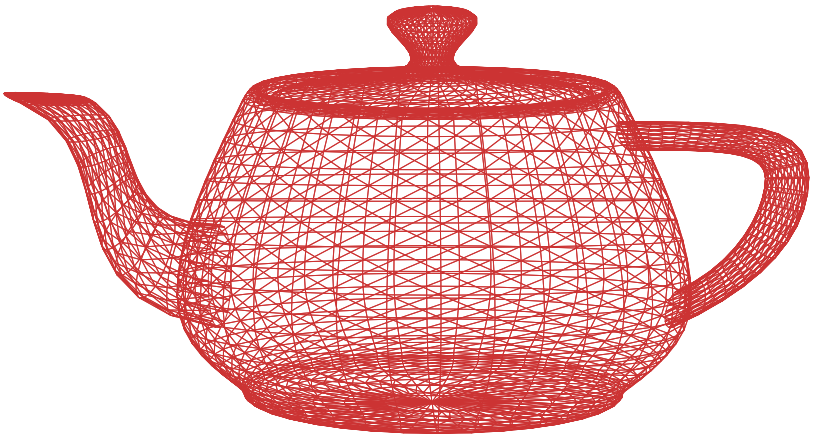
Useful when the model has no groups to distinguish.

Wireframe

Wireframe mode draws every triangle edge without fill or back-face culling, showing the full mesh topology. Control appearance with `wireframe: (color, width)`.

It's a great occasion to showcase SVG output, no rasterization artifacts and kind-of infinite zooming allowed, thanks to vectors.

```
#render-obj(teapot,
  up: (0, 1, 0),
  distance: 8,
  auto-fit: false,
  background: "#ffffff",
  wireframe: (color: "#cc3333", width: 0.1),
  mode: "wireframe",
  format: "svg",
  width: 55%,
)
```



Solid + Wireframe

Combines solid shading with wireframe edges overlaid on top. Useful for visualizing mesh density and triangle distribution while still seeing the shaded surface. Configure edge appearance with `wireframe: (color, width)`.

Here `antialias: 3` should be set, wireframe's strokes benefit from antialiasing.

```
#render-obj(teapot,
  up: (0, 1, 0),
  distance: 8,
  antialias: 3,
  mode: "solid+wireframe",
  wireframe: (width: 0.3),
  width: 80%,
  zoom: 1.2,
)
```

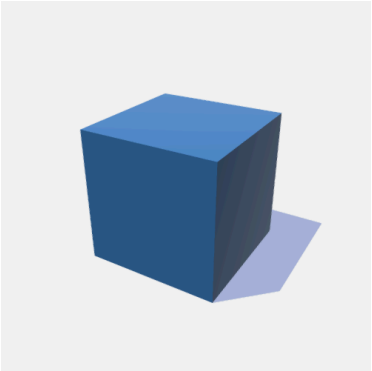


Shadows

Ground Shadow

A ground shadow is cast by projecting every triangle onto the ground plane along the light direction. Pass `ground-shadow: true` for defaults, or customize:

```
#render-stl(cube,
  camera: (3, 2, 2),
  light-dir: (1, -1, 3),
  ground-shadow: (opacity: 0.35, color: "#2244aa"),
  width: 50%,
)
```



Cast Shadows

PNG ONLY `shadows` renders true **self-shadowing**, where every part can occlude every other, computed with a depth map per light.

Option	Default	What it does
`per-pixel`	`false`	Sample shadows per fragment instead of per vertex, for crisp edges on low-poly and CAD models. PNG only, ~2.5× the cost, and required by `light-size` and `color` below.
`light-size`	`0`	Light radius in world units (alias `pcss-light-size`); area lights use their own `size` instead. Any value `> 0` enables the PCSS soft shadows described above: sharp where parts touch, blurring with distance.
`color`	`""`	Hex tint for the shadowed regions instead of darkening toward neutral grey (e.g. a cool blue).
`strength`	`1.0`	How dark shadows go, from `0` (none) to `1` (removes all direct light).
`softness`	`1`	PCF (percentage-closer filtering) blur radius, in texels: `0` = hard edges, `1` = 3×3, higher = softer everywhere.
`resolution`	`512`	Shadow-map size per light (res × res texels). Higher is sharper but slower to build.
`omni`	`false`	Render six cube-map faces so a positional light **inside** the geometry casts in every direction. ~6× the cost.
`bias`	`0.0008`	Constant depth-compare bias, the baseline fix for shadow acne (self-shadowing speckle).
`normal-bias`	`2.0`	Offsets the sample along the surface normal (in texels); the primary acne fix.
`slope-bias`	`1.0`	Adds extra bias on surfaces lit at a grazing angle, where acne is worst.

`cast-shadow` is a per-light option, set inside the `lights` array (see Multi-Light), not a `shadows` one.



`shadows: false (default)`



`shadows: (per-pixel: true, light-size: 8, …)`

Post-Processing

Antialiasing

PNG ONLY The `antialias` parameter is the single antialiasing control for PNG output; SVG is vector, so it needs none.

- `0`: off.
- `1`: a fast **FXAA** edge-smoothing pass, without supersampling. The quickest option when compile time matters most.
- `2`: renders at 2×2 the resolution and downsamples.
- `3`: renders at 4×4 the resolution, the highest quality. Best for wireframe and stroke, where straight edges benefit most.
- `4` (default) and `5`: FXAA on top of 2× and 4× supersampling, to smooth the slight steps supersampling leaves.

Supersampling covers geometry, shading and outlines. Ambient occlusion, depth fog, bloom, glow and sharpening run on the downsampled image, so their cost does not grow with the supersampling factor.

No antialiasing (`antialias: 0`)



FXAA (`antialias: 1`)



FXAA + 2× supersampling (`antialias: 4`, default)



4× supersampling (`antialias: 3`)



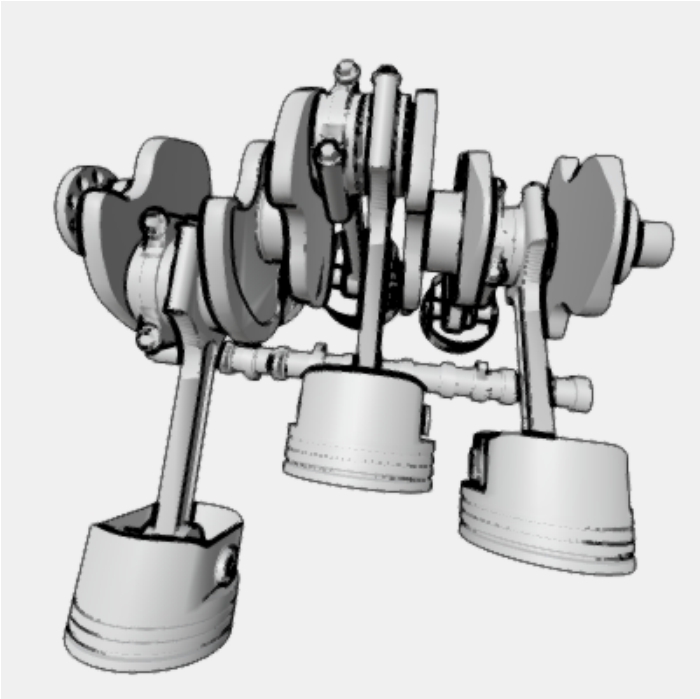
Silhouette Outlines

Draws bold edges along the model's silhouette and wherever the depth buffer jumps or folds: the rim of the lid, a limb in front of the body, the gap between two parts. Configure with `outline: true` for defaults, or customize:

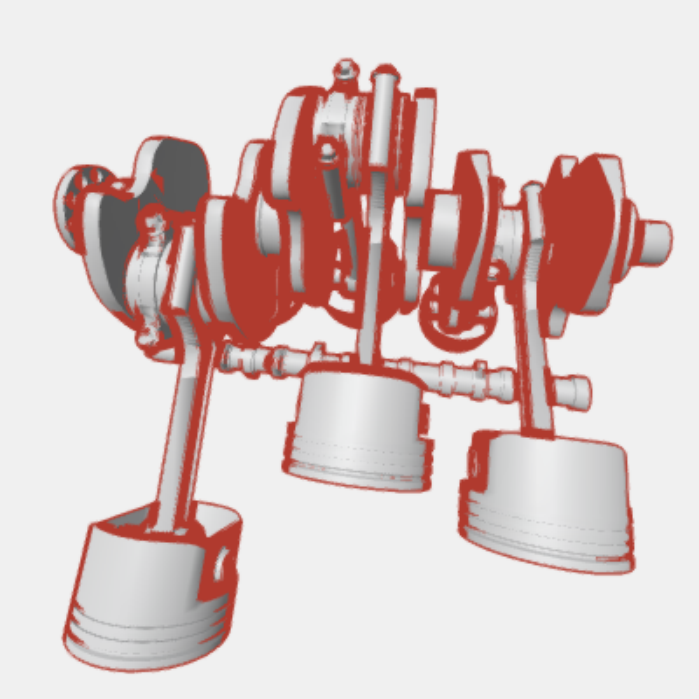
```
outline: (color: "#000000", width: 2, threshold: 5)
```

`width` is in output pixels. A depth jump counts as an edge when it exceeds `threshold` pixels' worth of scene distance at that depth; the default `5` catches anything steeper than a ~79° slope. Measured this way, the same edges appear at any zoom, distance, image size or model scale; lower values also catch shallower steps. Folds are found from the surface's true slope. `threshold: none` restores the older rule (a jump over 1.5% of the distance to the camera). In SVG output, outlines are traced from the mesh's silhouette edges instead, and `threshold` has no effect.

Defaults (`outline: true`)



`outline: (color: "#b03a2e", width: 3, threshold: 2)`



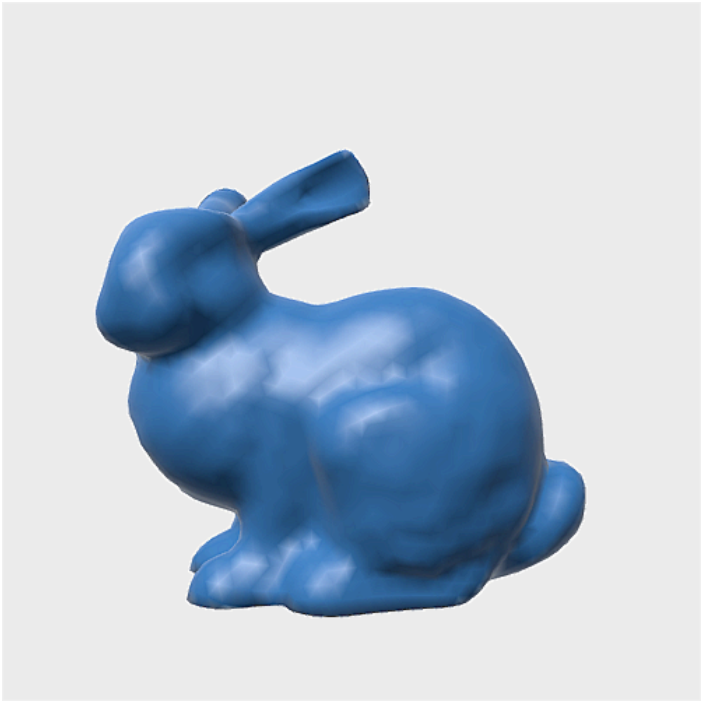
Sharpening

PNG ONLY Sharpening enhances edge contrast using a 3×3 unsharp mask. Pass `sharpen: true` for default strength (0.5), or customize with `sharpen: (strength: N)`. Higher values produce a more pronounced effect.

Without



Sharpen (strength: 2)

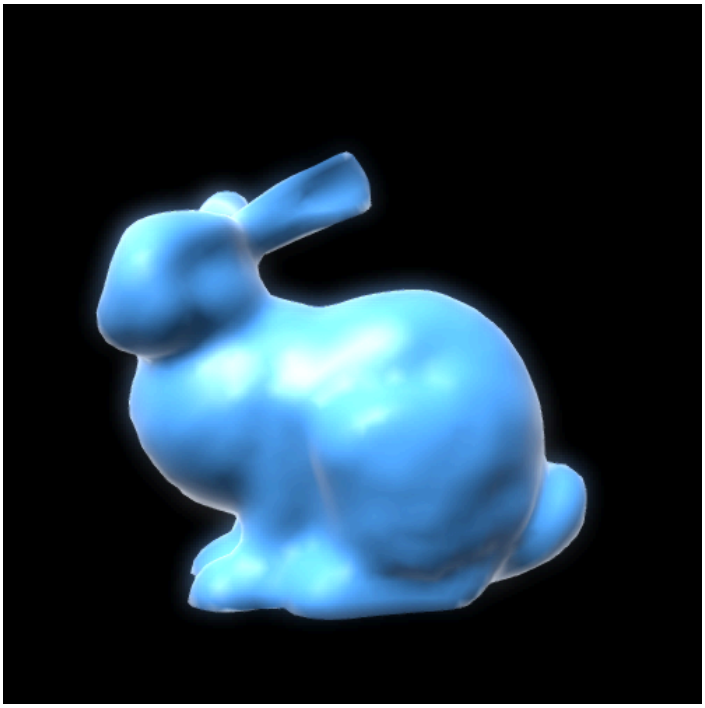


Bloom & Glow

PNG ONLY Bloom makes bright areas bleed light outward. Glow creates a uniform aura around the model's silhouette. Use `bloom: true` / `glow: true` for defaults, or customize:

```
bloom: (threshold: 0.8, intensity: 0.3, radius: 10)
glow: (color: "#ffffff", intensity: 0.5, radius: 15)
```

Bloom



Glow



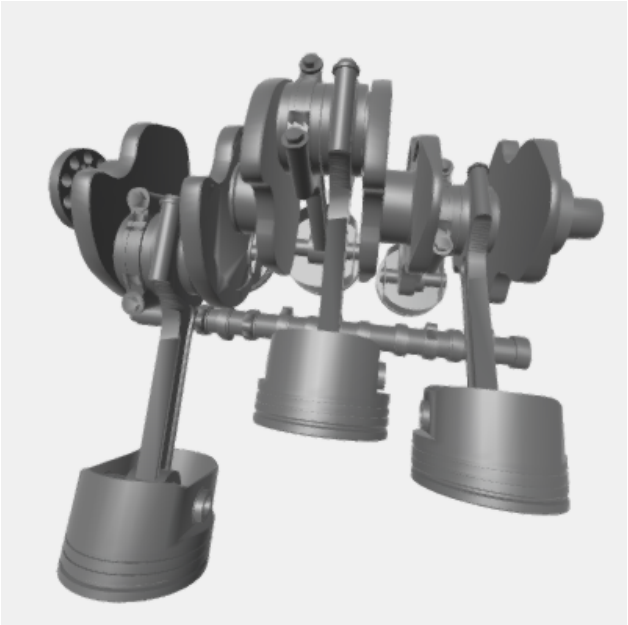
Ambient Occlusion

PNG ONLY Ambient occlusion darkens crevices and places where surfaces nearly touch. Use `ssao: true` for defaults, or tune it:

```
ssao: (samples: 16, radius: 20, bias: 0.025, strength: 1.0, space: "scene")
```

- **`radius`**: neighborhood size in model units, 10% of the bounding radius by default.
- **`bias`**: ignores blockers closer than this fraction of `radius`, to avoid self-shadowing.
- **`strength`**: how dark occluded areas get. **`samples`**: more is smoother.
- **`space`**: `"scene"` (default) or `"screen"`, where `radius` is a fraction of the image.

Without SSAO



With SSAO



Depth Fog

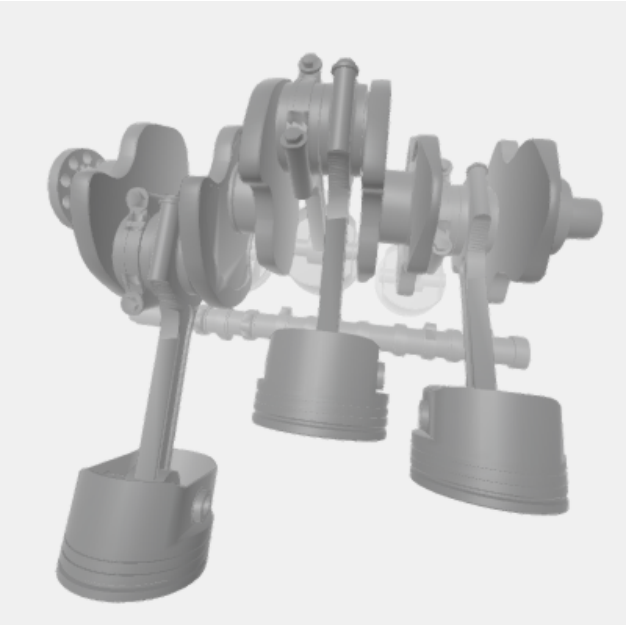
PNG ONLY Depth fog fades surfaces toward the background the farther they are from the camera. `intensity` (0–100) sets where the fade starts, from the back of the model (`0`, no fog) to its front (`100`):

```
fog: (intensity: 50, color: "#ffffff")  // color defaults to the background (white if transparent)
```

Without fog



With fog (`fog: (intensity: 100)`)



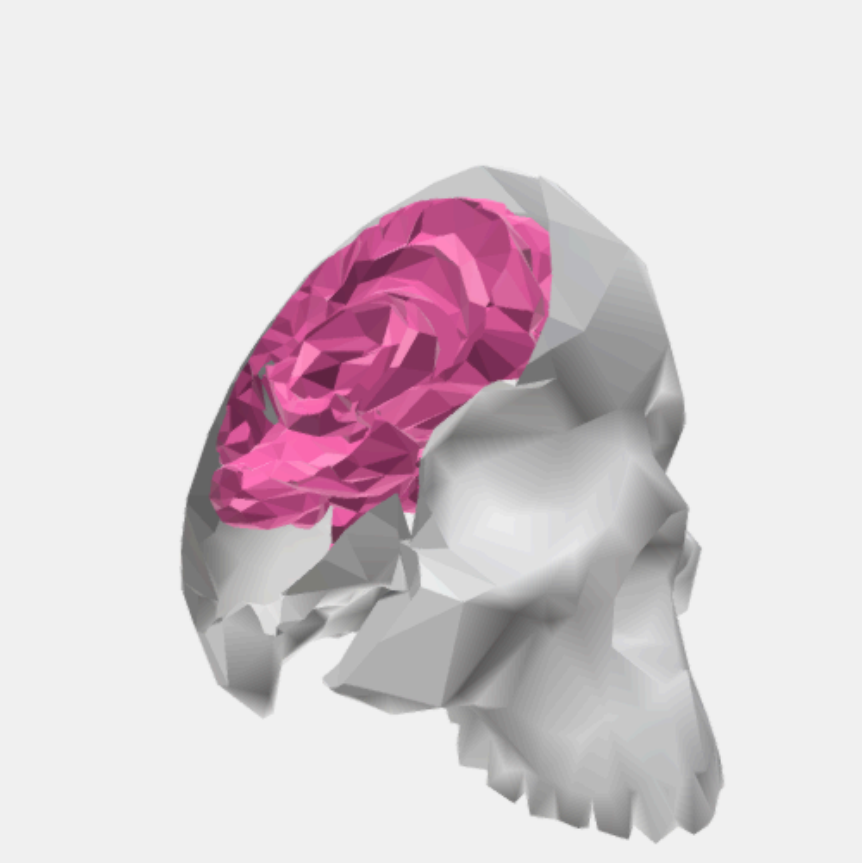
Effects

Clipping

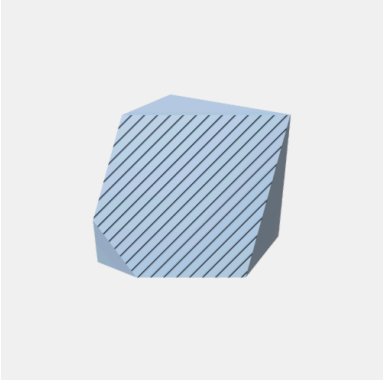
Slice the model with a plane to cut part of it away, for section drawings or to reveal internal geometry. `clip` takes either an explicit world-space plane `(a, b, c, d)`, keeping the `ax + by + cz + d >= 0` half, or a dictionary that positions the plane for you.

Wrapping the plane in a dict lets you add `cap: false`, which leaves the cross-section open so you can see inside. Here an explicit plane opens the skull to reveal the brain:

```
#render-obj(skull-brain,
  azimuth: 220,
  up: (0, 1, 0),
  distance: 180,
  highlight:
    ("Skull": (color: "#e8e8e8"),
     "Brain": (color: "#ff69b4")),
  clip: (plane: (2, -1, 0, 1), cap: false),
  cull-backface: false,
)
```



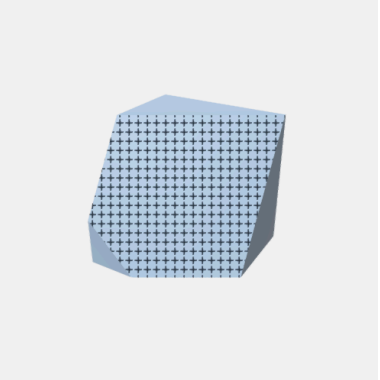
Cap it instead (the default) and the cut face can be **hatched** for an engineering-style section view. Three patterns, parallel `"lines"`, a `"cross"` grid and discrete `"crosses"`, each honour `angle`, `spacing`, `width`, and `color` (the whole pattern rotates with `angle`), in SVG and PNG alike:



`style: "lines", angle: 45`



`style: "cross", angle: 30`



`style: "crosses", angle: 0`

The dict form positions the plane and controls the cut:

- **`from: "camera"`** squares the plane to the view direction, so the slice always faces the viewer whatever the angle (or use `axis: "x"/"y"/"z"`, `normal: (x, y, z)`, or a raw `(a, b, c, d)` plane).
- **`depth`** sets how deep to cut, from `0` (near face) to `1` (far); `distance` sets it in world units instead.
- **`keep`** chooses which half to keep: `"far"` (default) or `"near"`.
- **`cap`** closes the cross-section with a flat face (default `true`); `false` reveals hollow interiors, as in the skull above.
- **`hatch`** draws section lines over the cap: `true` for defaults, or a dict of `style` (`"lines"`/`"cross"`/`"crosses"`), `angle`, `spacing`, `width`, and `color`. Needs `cap: true`.

Exploded View

Move model parts outward from the model center. Very useful to showcase different parts of a system in mechanics.

For OBJ files with `g` or `o` groups, each group is treated as a separate component.

```
#render-obj(crankshaft,
  up: (0, -1, 0),
  camera: (-200, -200, 500),
  color: "#555555",
  explode: 0.8,
)
```



For PLY, STL files or OBJ files without groups, connected components are detected automatically using shared edges (union-find). Each component is then offset by `explode * (component_centroid - model_center)`.

This is the case for this exploded teapot.

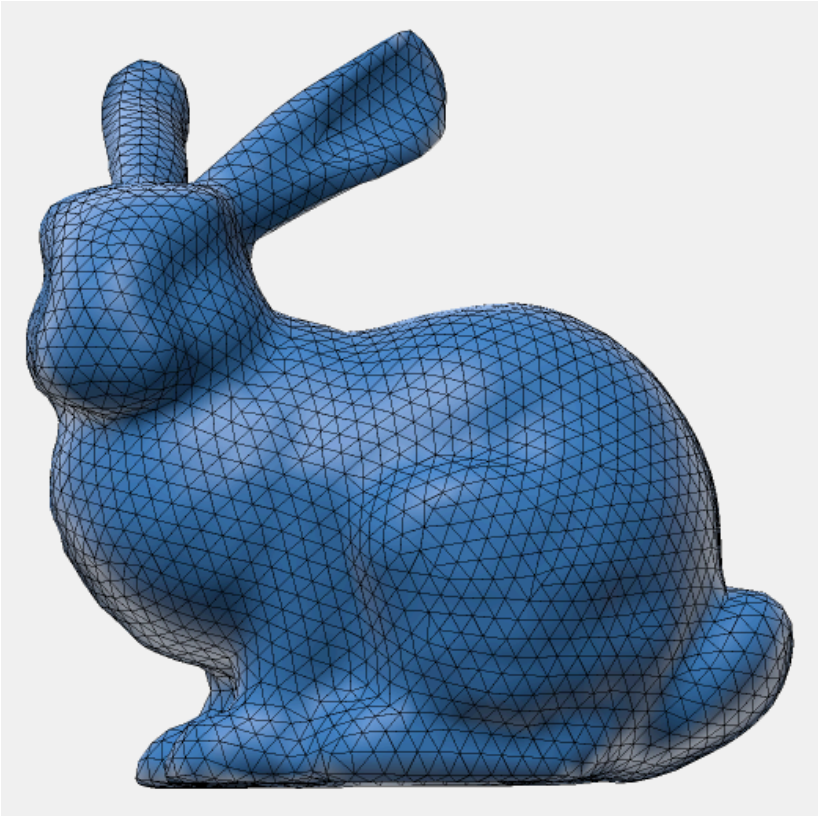
```
#render-obj(teapot,
  camera: (0, 2, 5),
  up: (0, 1, 0),
  explode: 0.5,
)
```



Decimation

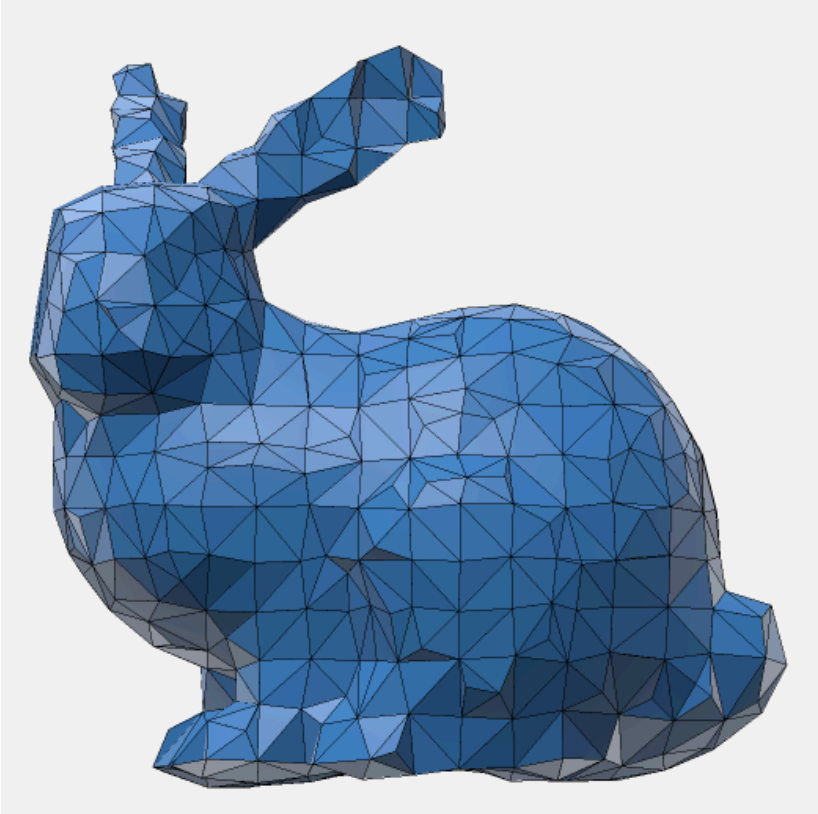
Reduce a mesh's triangle count with grid vertex clustering, a fast, format-agnostic mesh simplification that applies identically to STL, OBJ and PLY. It is handy for shrinking dense scans or high-poly exports so they render (and embed in your PDF) faster. It pairs well with the default smooth shading, which re-derives vertex normals and softens the faceting.

The `decimate` strength runs from `0` (off, default) to `1` (most aggressive): a uniform grid is laid over the model, vertices sharing a cell collapse into one, and higher values use a coarser grid that merges more detail. The wireframe overlay below makes the thinning topology visible.



Original (`decimate: 0`)

Vertices: **3301**
Triangles: **6598**



`decimate: 0.75`

Vertices: **633**
Triangles: **1284**

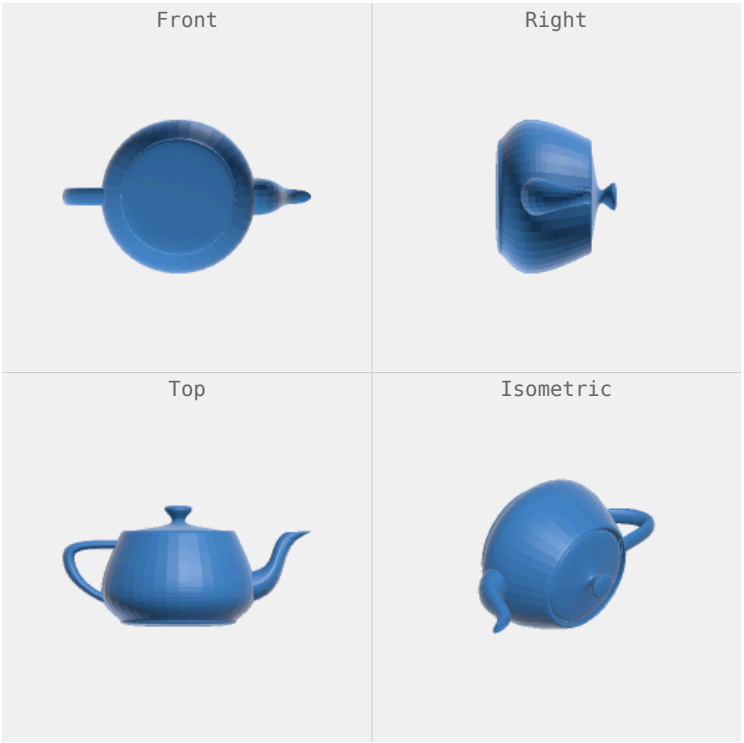
Fewer triangles means faster rendering, and the default smooth shading re-derives vertex normals afterward, so moderate decimation stays visually clean.

Multi-View

Multi-View Grid

Render multiple named views in a single image, similar to an engineering drawing sheet. Available views are `"front"`, `"back"`, `"left"`, `"right"`, `"top"`, `"bottom"`, and `"isometric"`. The renderer arranges them in a grid and labels each cell.

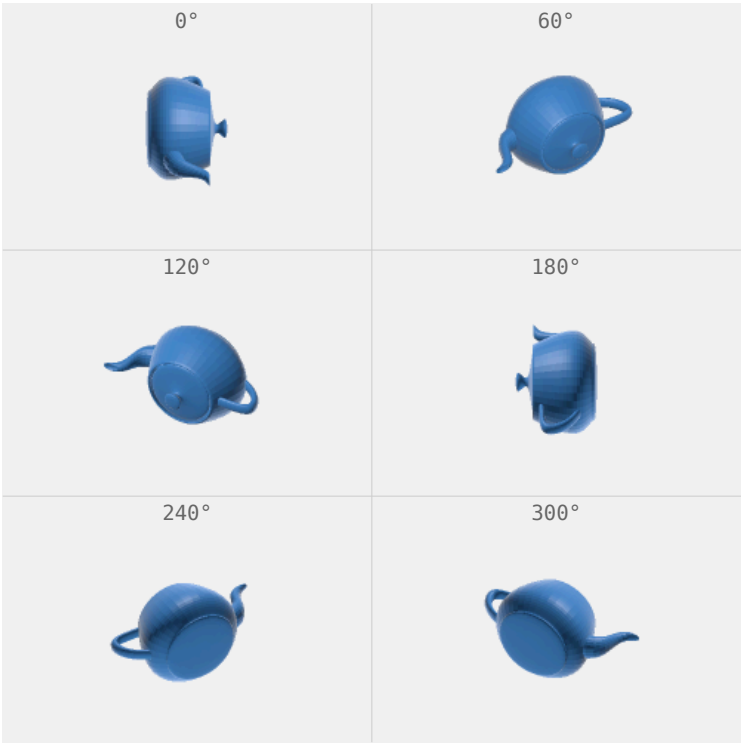
```
#render-obj(teapot,
  views: ("front", "right", "top", "isometric"),
  grid-labels: true,
  cull-backface: false,
)
```



Turntable

Automatically generates a grid of views evenly spaced around the model at a fixed elevation angle. Use `turntable: (iterations: 6, elevation: 40)` or just `turntable: 6` for the number of views. View labels showing the azimuth angle are displayed by default; set `grid-labels: false` to hide them.

```
#render-obj(teapot,
  turntable: (iterations: 6, elevation: 40),
  grid-labels: true,
  cull-backface: false,
)
```

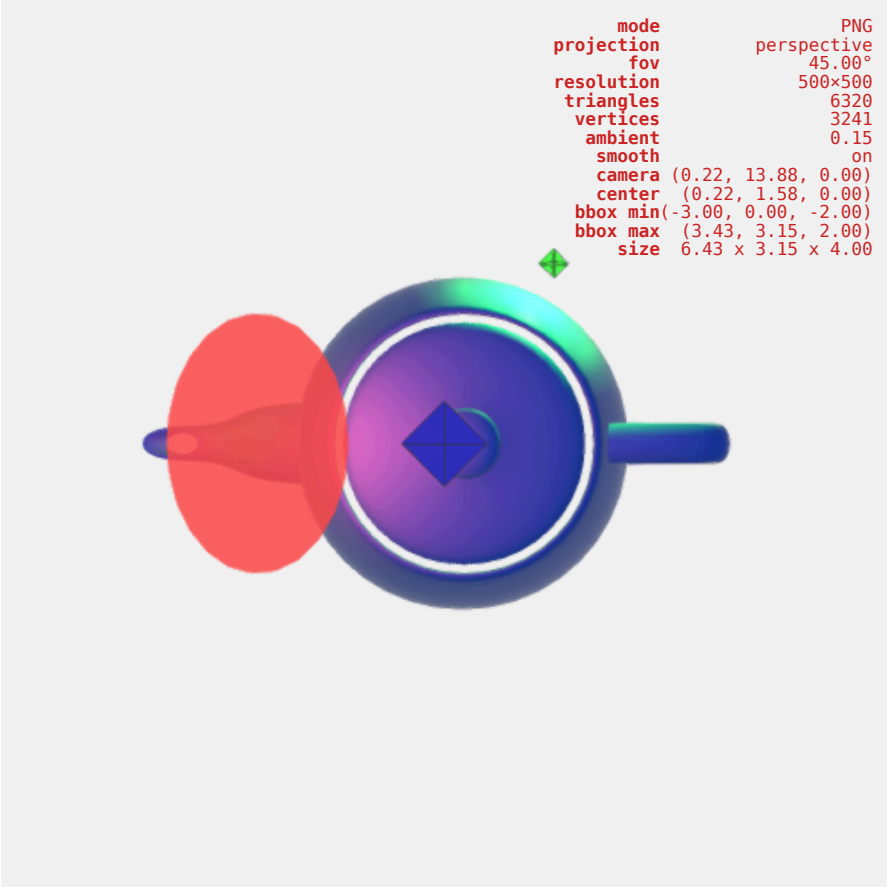


Debug

`debug: true` overlays model metadata (triangle count, bounding box, camera position) directly on its canvas. The overlay text is drawn in `debug-color` (default `#cc2222`); set it to keep the labels legible against your model or background.

It also renders lights as octahedrons of the color they emit, to allow placing lights seamlessly around your model. Area lights (those with a `size`) are drawn instead as a disk of that radius, facing the model, so you can gauge their extent.

```
#render-obj(teapot,
  debug: true,
  lights: (
    (type: "area",
     vector: (2, 4, 0),
     size: 1.2,
     color: "#ff4444",
     intensity: 1.2),
    (type: "positional",
     vector: (-1, 2, 2),
     color: "#44ff44",
     intensity: 1.0),
    (type: "directional",
     vector: (0, 1, 0),
     color: "#4444ff",
     intensity: 0.5),
  ),
)
```



Model Info & Measurements

`get-stl-info`, `get-obj-info`, and `get-ply-info` return a dictionary of the model's metadata and geometric measurements, all in the file's own units. Alongside `triangles`, `vertices`, the bounding box (`bbox_min`/`bbox_max`/`bbox_center`/`bbox_radius`), and the resolved `camera`/`center`/`projection`/`fov`, it reports:

Field	What it is
`size`	Bounding-box dimensions `(dx, dy, dz)`.
`surface_area`	Total triangle area (exact).
`volume`	Enclosed volume: exact for a closed, consistently wound mesh, approximate for open or non-manifold ones.
`centroid`	Centre of mass `(x, y, z)` (volume-weighted).

Because it's a plain dictionary, you can drive labels, callouts, or camera framing from real geometry. Here it is run on the bunny:

```
#let info = get-obj-info(bunny)
#grid(columns: 2, column-gutter: 1em, row-gutter: 0.3em, align: (right, left),
  ..for (key, val) in info.pairs() { (strong(key), repr(val)) })
```

triangles	6598
vertices	3301
bbox_min	(-0.0945, 0.0332, -0.0614)
bbox_max	(0.0609, 0.1871, 0.0584)
bbox_center	(-0.0168, 0.1102, -0.0015)
bbox_radius	0.1247
size	(0.1555, 0.1539, 0.1199)
surface_area	0.0569
volume	0.0008
centroid	(-0.0209, 0.0868, 0.011)
camera	(-0.0168, 0.4843, -0.0015)
center	(-0.0168, 0.1102, -0.0015)
projection	"perspective"
fov	45.0

Models Credits

- Utah teapot, Stanford
- Stanford bunny, Stanford
- Crankshaft with pistons, CGTrader
- Low-poly brain, Sketchfab
- Low-poly skull, Printables
- Rubik's cubes: Blender generated & LiDAR scanned by myself
- Blue Marble Earth texture, NASA (public domain); globe mesh generated for this example
- Treasure Chest, Rico Cilliers / Poly Haven (CC0)

Contributing

Bug reports, feature requests, issues, new feature ideas are welcome on GitHub.